

Beginner Guide to Managing Environments with Terraform Workspaces

Terraform workspaces let you manage multiple environments like Dev, Staging, and Production from a single codebase. Instead of duplicating code files, workspaces isolate state management so you can deploy identical infrastructure safely across stages.

What you'll learn:

- 1 Creating and Switching Environments
- 2 Verifying Your Active Workspace
- 3 Dynamic Resource Naming in Code
- 4 Cleaning Up Unused Workspaces

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Creating and Switching Environments

Workspaces isolate state files so resources in dev do not mix with prod. Creating a new workspace creates an isolated state path inside your backend storage. Switching workspaces instantly directs all plan and apply commands to that target state without modifying your HCL code.

```
$terraform workspace new dev  
terraform workspace select dev
```

Cloud & DevOps Daily

Terraform Workspaces: Manage Dev, Staging & Prod Easily

Monday, September 14, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Verifying Your Active Workspace

Operating in the wrong environment is the most common Terraform rookie mistake. Running verification commands gives you immediate visual confirmation of where changes will be applied. Get into the habit of checking your active workspace context before every single execution.

```
$terraform workspace show
```

Cloud & DevOps Daily

Terraform Workspaces: Manage Dev, Staging & Prod Easily

Monday, September 14, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Dynamic Resource Naming in Code

You can reference the built-in variable string `terraform.workspace` directly inside configuration blocks. This dynamically names resources according to the active workspace name. It keeps your code DRY while preventing naming collisions between environments across cloud providers.

```
$resource "aws_s3_bucket" "app_storage" {  
  bucket = "my-company-app-${terraform.workspace}-data"  
}
```

Cloud & DevOps Daily

Terraform Workspaces: Manage Dev, Staging & Prod Easily

Monday, September 14, 2026

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Cleaning Up Unused Workspaces

Temporary test environments clutter state storage if left unmanaged after testing ends. Before removing a workspace, run destroy to release managed cloud resources. Terraform automatically prevents workspace deletion if resources are still tracked inside that state file.

```
$ terraform destroy
terraform workspace select default
terraform workspace delete dev
```

Cloud & DevOps Daily

Terraform Workspaces: Manage Dev, Staging & Prod Easily

Monday, September 14, 2026

See Key Takeaways on next page >> linkedin.com/in/mrmanrajsingh

Key Takeaways

- + Always verify active context with 'terraform workspace show' before running apply.
- + Use 'terraform.workspace' inside resource tags for clear cloud cost allocation.
- + Pair workspaces with environment-specific var files like dev.tfvars and prod.tfvars.
- + Keep default workspace clean and use explicit named workspaces for active environments.
- + Use separate AWS accounts alongside workspaces for strict production security boundaries.

Watch Out For

- ! Running 'terraform destroy' while accidentally switched to the production workspace.
- ! Hardcoding environment names in HCL instead of using dynamic workspace variables.
- ! Attempting to delete an active workspace before switching away to another workspace.
- ! Assuming workspaces provide complete access isolation between user permissions.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mrmanrajsingh